Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

CS101

From pieces to flow

Today
Welcome to your next lesson: From pieces to flow.

I'm getting your personalized lesson ready, using what you've already learned, what's coming up next, and the best way to help things click.

Step by step, you're moving forward.

Imagine an online shop checkout 🛒

You start with a base price, then:

first add tax
then apply a discount

In code, that looks like a flow where the output of one function becomes the input of the next:

price → add_tax() → apply_discount() → final total

We'll build this kind of pipeline in Python, step by step.

Quick check, Ron: does that real-world checkout flow make sense as a way to think about connecting functions together? 🙂

Yes, that makes sense

Nice 👍

Let's zoom in on just one step in that flow.

Say we have a function that adds tax:

Python

```
def add_tax(price):
```

```
    tax_rate = 0.08   # 8% tax
    total_with_tax = price * (1 + tax_rate)
    return total_with_tax
```

Here’s the key idea:

You give add_tax a price, and it returns a new value (total_with_tax) that you can use in the next step.

So if you do:

Python

```
price_after_tax = add_tax(100)
```

Then price_after_tax is now ready to be passed into another function (like apply_discount).

Does this step—“use the return value from add_tax as the next input”—feel clear so far?

Yes, it's clear

Great 👍

Next small idea: you can chain two functions so the return value of the first goes directly into the second.

Visually:

original_price → add_tax → apply_discount → final_total

In Python, there are two common styles:

Using an intermediate variable (very clear):

```
price_after_tax = add_tax(100)
final_price = apply_discount(price_after_tax)
```

Nesting calls (shorter, but a bit harder to read):

```
final_price = apply_discount(add_tax(100))
```

Both do the same thing: the result of add_tax(100) is used as the input to apply_discount.

Before we write your own version:

Which style feels clearer to you right now — with a middle variable, or the fully nested one?

2nd i used t earlier i code

Nice, so the nested style feels natural to you 👍

Let’s use that.

We’ll build a tiny pipeline:

base price → add_tax → add_shipping → final total

Step by step:

You already saw add_tax(price)
Now we add another helper: add_shipping(price)
Then we chain them like:
final_total = add_shipping(add_tax(base_price))
Your turn in the code editor 🔒

In the editor, please:

Define add_tax(price) with an 8% tax (use 0.08, not 8)
Define add_shipping(price) that adds a flat 5 dollars
In your main code, set base_price = 100
Compute final_total using the nested style:
final_total = add_shipping(add_tax(base_price))
print(final_total)

Run it, then hit Submit code, and I'll check it with you.

Python

```
def add_tax(price):
    tax_rate = 0.08   # 8% tax
    total_with_tax = price * (1 + tax_rate)
    return total_with_tax

def add_shipping(price):
    return price + 5

def main():
    base_price = 100
    final_total = add_shipping(add_tax(base_price))
    print(final_total)

if __name__ == "__main__":
    main()
```

Ron, this is excellent ☑

add_tax is correct (8% written as 0.08)
add_shipping is simple and clear
The chaining here is exactly what this lesson is about:
final_total = add_shipping(add_tax(base_price))
Output 113.0 is right:
100 → add 8% tax = 108 → add 5 shipping = 113
Now I want to lock in one tiny but important idea:

When we say "8%" in math, in code we must use 0.08, not 8.

I'll give you a quick one-question check on percent vs decimal next — ready for a tiny multiple-choice question on that? 🎯

i just opie your uctio sorry

No worries at all, Ron — that's actually what I wanted you to do here 👀

The important part is:

You copied the structure
You understood it enough to run it
And it produced the right result

Now let’s make sure the percent vs decimal piece is solid, because it bites a lot of people in price/tax code.

Quick mental check (don’t run code, just think it through):

If you want to apply a 15% discount in Python, which number should you multiply by?

A) 0.15

B) 15

You can just answer with A or B.

Python

```
def add_tax(price):
    return price * 1.08

def add_shipping(price):
    return price + 5

def main():
    base_price = 100
    final_total = add_shipping(add_tax(base_price))
    print(final_total)

if __name__ == "__main__":
    main()
```

This version looks great too, Ron

You simplified add_tax:

Python

```
def add_tax(price):
    return price * 1.08   # same as price * (1 + 0.08)
```

That still means “add 8%” because 1.08 = 1 + 0.08.

Your pipeline is:

Python

```
final_total = add_shipping(add_tax(base_price))
```

So the flow is:

100 → add_tax → 108.0
108.0 → add_shipping → 113.0 ☑
Next small step: I’d like you to extend this pipeline with one more function, like a discount or coupon.

Which would you rather add?

a percentage discount (e.g. 10% off), or
a fixed coupon amount (e.g. $7 off)?
discount

Perfect, let’s add a percentage discount

We'll keep the same flow, just one more step at the end:

base price → add_tax → add_shipping → apply_discount → final total

We'll say the discount is 10% off.

So we want a helper like:

Python

```
def apply_discount(price):
    # 10% off → keep 90% → multiply by 0.90
    return price * 0.90
```

Key detail:

10% = 0.10, so you keep 1 - 0.10 = 0.90 of the price.
Your turn in the code editor:

Add apply_discount(price) with a 10% discount (use 0.10 or 0.90 correctly).

Update main() so the chain is nested:

```
final_total = apply_discount(add_shipping(add_tax(base_price)))
print(final_total).
```

Run, then Submit code, and I'll check both your math and your chaining.

Python

```
def add_tax(price):
    return price * 1.08

def add_shipping(price):
    return price + 5

def apply_discount(price):
    return price * .90

def main():
    base_price = 100
    final_total = apply_discount(add_shipping(add_tax(base_price)))
    print(final_total)

if __name__ == "__main__":
    main()
```

This is spot on, Ron 🔥

Your pipeline is now:

Python

```
final_total = apply_discount(add_shipping(add_tax(base_price)))
```

Data flow:

add_tax(100) → 108.0
add_shipping(108.0) → 113.0
apply_discount(113.0) → 113.0 * 0.90 = 101.7 ☑

You've just built a full checkout pipeline where the return value of each function becomes the input of the next. That's exactly the mastery goal for this lesson.

Quick check-in:

Do you feel comfortable now with this idea of chaining functions like this, or want one more tiny practice example before we wrap the lesson?

I'm comfortable with it

Love it. You earned that confidence here 👌

Quick recap of what you just did:

Wrote small, single-purpose helpers: add_tax, add_shipping, apply_discount
Chained them so each return value became the next input:
final_total = apply_discount(add_shipping(add_tax(base_price)))
Handled percentages correctly using decimals (0.08, 0.90, etc.)
Built a real-world style checkout flow from scratch 🛒
You've fully hit the goal for this lesson.

If you want to go further on your own, a fun tweak is to play with the order, like:

discount before tax, or
different tax/discount values,
and see how the final number changes.

For now, you're all set to move on to the next lesson when you're read

```
def add_tax(price):
    return price * 1.08

def add_shipping(price):
    return price + 5

def apply_discount(price):
    return price * .90

def main():
    base_price = 100
    final_total = apply_discount(add_shipping(add_tax(base_price)))
    print(final_total)

if __name__ == "__main__":
    main()
```